

# SecureLink

## Privacy-Preserving Record Linkage with XGBoost

Project Highlight

---

**Steven Filbert** | Developer (XGBoost model owner)

Macquarie University School of Computing • COMP3850 Computing Industry Project • S1 2025 • Group 46 (Cybersecurity stream)

### Project Overview

---

SecureLink is a privacy-preserving record linkage (PPRL) system that matches records belonging to the same person across two separate datasets without exposing the underlying sensitive data. It was built for a healthcare context, where two parties may hold records about the same patient but cannot legally share raw clinical text with one another.

The system lets the data be matched in an encoded, noise-protected form rather than in the clear, using Bloom filter encoding and Local Differential Privacy (LDP). Five linkage models were explored across the team; this document covers the XGBoost component, which I designed, built, integrated, and evaluated.

### My Contribution

---

I owned the XGBoost prototype end to end and trained it as a binary classifier (match vs non-match) under three configurations, in order to measure the real cost of adding privacy:

- **Raw baseline.** Text converted to features with TF-IDF, reduced to 200 dimensions with Truncated SVD. No privacy applied, used as the performance benchmark.
- **Local Differential Privacy (LDP).** Calibrated random noise added to the feature vectors, with noise strength controlled by a privacy budget ( $\epsilon$ ).
- **Bloom filter encoding.** Records hashed into bit vectors and compared with Dice similarity, using that similarity score as the model input.

*I also integrated the model into the team's React dashboard so results could be run and viewed interactively.*

## Key Results

Measured on the real project dataset ( $\epsilon = 7$ ). Adding LDP privacy cost only about 1.7% in F1 versus the raw baseline, showing strong privacy protection at minimal accuracy loss. The Bloom filter approach traded more recall for its privacy guarantees.

| Configuration                           | F1            | Accuracy | Precision | Recall        |
|-----------------------------------------|---------------|----------|-----------|---------------|
| Raw XGBoost (baseline)                  | <b>0.9118</b> | 0.9131   | 0.9237    | 0.9003        |
| LDP XGBoost ( $\epsilon = 7$ )          | <b>0.8960</b> | 0.8980   | 0.9118    | 0.8808        |
| Bloom Filter XGBoost ( $\epsilon = 7$ ) | <b>0.7252</b> | 0.8837   | 0.8754    | <b>0.6189</b> |

*Real project results. Source: project documentation (non-confidential).*

## Demonstration (Synthetic Data)

To illustrate the approach safely, the pipeline was re-run on randomly generated dummy data that contains no real records and no personal information. The pattern matches the real project: each privacy step costs some performance, and the Bloom filter loses the most on recall.

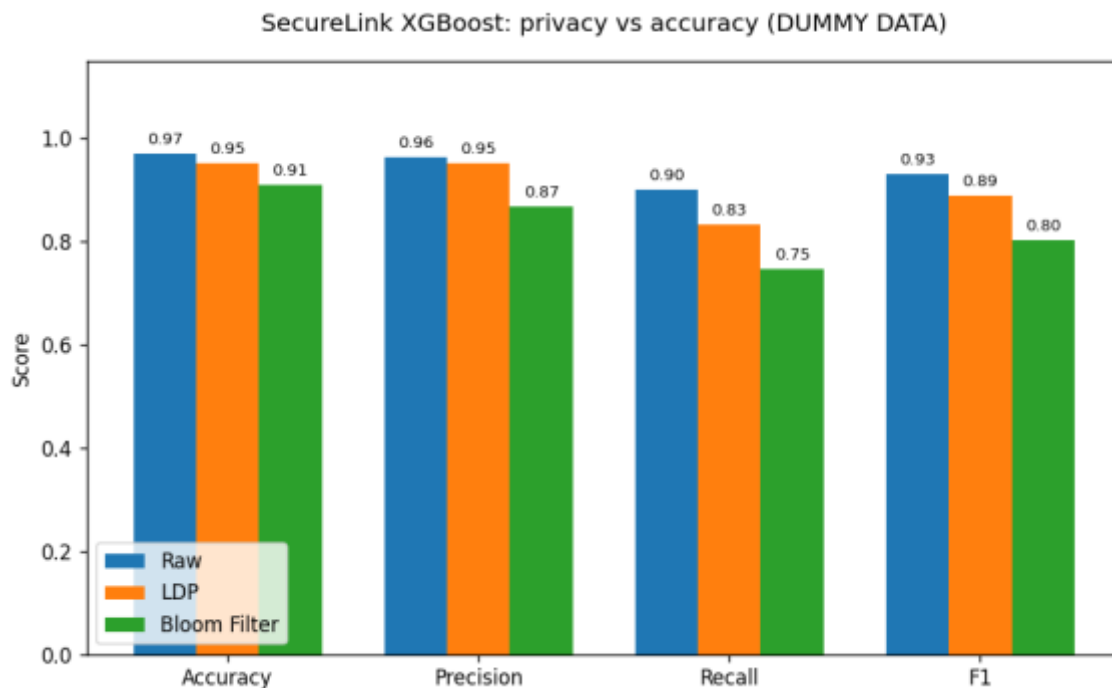


Figure 1. Model comparison across the three privacy configurations (synthetic data).

SecureLink: sample matches and non-matches (DUMMY DATA)

| Type      | Note A (text1)                           | Note B (text2)                           | Dice sim |
|-----------|------------------------------------------|------------------------------------------|----------|
| Match     | fracture chronic with stable review p... | presents prescribed review with patie... | 0.894    |
| Match     | chronic acute reflux chronic follow i... | acute ibuprofen chronic follow reflux... | 0.878    |
| Match     | prescribed of chronic with eczema of ... | salbutamol eczema prescribed mild his... | 0.897    |
| Match     | ventolin patient history presents mig... | patient with presents history ventoli... | 0.864    |
| Match     | with mild patient mild acute patient ... | asthma patient prescribed patient acu... | 0.872    |
| Non-match | aspirin presents patient up prescribe... | acute salbutamol chronic prescribed r... | 0.763    |
| Non-match | chronic with with patient history pre... | history presents prescribed with pres... | 0.804    |
| Non-match | chronic atorvastatin with of of prese... | up review patient paracetamol mild of... | 0.786    |
| Non-match | patient history prescribed of paracet... | prescribed patient with chronic prese... | 0.781    |
| Non-match | prescribed eczema with prescribed sal... | history mild patient prescribed stabl... | 0.743    |

Figure 2. Sample matched and non-matched record pairs with Dice similarity scores (synthetic data).

## Model Design and Decisions

---

The task was framed as binary classification: for each pair of records, predict whether they refer to the same entity (match) or not (non-match). Because real linkage data contains far more non-matching pairs than true matches, the Bloom filter dataset was rebalanced to a 3:1 ratio of non-matches to matches before training, so the model would not simply default to predicting “non-match”.

Feature design differed by configuration. The raw and LDP models used 200 numeric features (100 Truncated SVD components from each of the two text fields), while the Bloom filter model used a single feature: the Dice similarity between the two encoded records. All configurations were trained on over a thousand record pairs and evaluated on a 70/30 train/test split with a fixed random seed, reported on Accuracy, Precision, Recall, and F1.

XGBoost was run with default hyperparameters as a deliberate baseline, since the aim was to compare the three privacy configurations on equal footing rather than chase peak accuracy. A clear next step would be tuning `scale_pos_weight` to reflect the class imbalance, alongside `max_depth`, `learning_rate`, and `n_estimators` via grid or randomised search.

## Pipeline Pseudocode

---

A high-level view of the pipeline logic (no confidential code or data).

```
INPUT: train.csv, test.csv    # each row is a record pair: text1, text2, label
                                # label = 1 if same person, else 0
                                # privacy budget (smaller = more privacy)
    epsilon                    # privacy budget (smaller = more privacy)

# 1. Turn text into numbers
features = TF-IDF(text1, text2)    # weight informative words
features = TruncatedSVD(features, 100) # shrink to 100 numbers per field

# 2. Model A: Raw baseline (no privacy)
model_raw = XGBoost().train(features_train, labels_train)
evaluate(model_raw, test)

# 3. Model B: LDP (privacy by adding noise)
flip_prob = 1 / (1 + e^(epsilon / 2))    # smaller epsilon -> more noise
noisy = features + random_noise(flip_prob)
model_ldp = XGBoost().train(noisy_train, labels_train)
evaluate(model_ldp, test)

# 4. Model C: Bloom filter (privacy by hashing)
bits = encode each record into a Bloom filter
bits = add_privacy_noise(bits)    # flip some bits
```

```
for each pair of records in the same block:
    feature = Dice_similarity(bits_i, bits_j)
    label   = (same person?)

# handle imbalance: keep all matches, keep 3x as many non-matches
balanced = downsample_non_matches(ratio = 3 : 1)
split balanced -> 70% train / 30% test
model_bf = XGBoost().train(bf_train, bf_labels_train)
evaluate(model_bf, test)

# 5. Compare all three on Accuracy / Precision / Recall / F1
```

## Tools & Skills

---

Python, XGBoost, scikit-learn (TF-IDF, Truncated SVD), Bloom filter encoding, Local Differential Privacy, React (dashboard integration), Agile / GitHub / Kanban.

## Confidentiality

---

Source code, algorithms, and the sponsor datasets remain confidential to the university research project. This document describes the methodology and presents non-confidential results and synthetic-data demonstrations only.